

CAMPUS 5G Protocol Benchmark — Follow-up Results

Adding QUIC Transports and Scaling to 50 Devices

Samiullah Khairy

May 29, 2026

1 What this follows up on

In the previous report (May 26) I benchmarked three protocols — gRPC, Zenoh, and MQTT over TCP — across 144 runs and found that MQTT over TCP breaks completely under degraded 5G conditions: median RTT reached 32 seconds at $N = 10$ with 1% packet loss. Zenoh and gRPC both handled it fine, sitting at the 160 ms network floor.

That report ended with two open questions:

1. Can swapping TCP for QUIC fix MQTT? QUIC avoids TCP head-of-line blocking, which was identified as the root cause of the queue collapse.
2. How do the protocols scale beyond $N = 10$? The original sweep only went up to 10 devices.

This report answers both. I extended the benchmark to five protocol configurations and scaled up to $N = 50$ devices, running a total of 360 experiments.

2 What changed in the setup

The containerized testbed, netem impairment profiles, and measurement loop are identical to the previous report. The only additions are two new protocol configurations:

- **Zenoh-over-QUIC** — the same Zenoh router and client images, reconfigured to use QUIC (UDP) instead of TCP.
- **MQTT-over-QUIC** — a completely different stack from MQTT over TCP. The broker is NanoMQ (not Mosquitto) with a native QUIC listener. The clients are C binaries compiled against NanoSDK with MsQuic support, running asynchronous callbacks instead of the polling loop the Python `paho-mqtt` client uses. This eliminates the 45 ms polling floor entirely.

I also extended the device count sweep from $N \in \{1, 2, 5, 10\}$ to $N \in \{1, 2, 5, 10, 20, 50\}$, giving the full matrix:

$$5 \text{ protocols} \times 3 \text{ profiles} \times 6 \text{ device counts} \times 2 \text{ payloads} \times 2 \text{ rates} = \mathbf{360 \text{ runs}}$$

Message volume at scale

At $N = 20$ devices, 10 Hz, 30 s, the edge sends 6,000 commands and receives 6,000 acknowledgements — 12,000 application-level messages per run. For broker-based protocols (MQTT), this becomes **24,000 broker message transitions**.

At $N = 50$, the same calculation gives 15,000 commands and 15,000 acknowledgements — 30,000 application-level messages per run, or **60,000 broker message transitions**. This is the load that exposes the architectural limits of each approach.

3 Results

3.1 Baseline: clean network

Table 1: Baseline latency on clean network ($N = 1$, 100 B, 10 Hz)

Protocol	p50 (ms)	p95 (ms)	p99 (ms)	Mean (ms)
Zenoh-QUIC	2.64	2.89	3.00	2.37
Zenoh (TCP)	2.61	2.88	2.98	2.30
MQTT-QUIC	1.95	5.36	5.65	2.68
gRPC	3.29	3.65	4.24	2.74
MQTT (TCP)	44.02	45.97	46.19	43.90

The first thing that jumps out: MQTT-over-QUIC drops the baseline from **44 ms to 2 ms**. This confirms that the 45 ms floor I saw in the first report was entirely the `paho-mqtt` polling delay, not anything intrinsic to the MQTT protocol. The NanoSDK C client with asynchronous callbacks eliminates it completely.

Zenoh and Zenoh-QUIC are essentially identical at baseline — both sub-3 ms — which makes sense since on a clean local network QUIC doesn’t add anything over TCP.

At $N = 50$ on the clean network, Zenoh and Zenoh-QUIC stay below 5 ms (p95). MQTT-QUIC rises to ~ 20 ms at this scale, which is NanoMQ’s per-stream multiplexing showing minor overhead at high concurrency. Interestingly, MQTT TCP’s polling floor also disappears at $N = 50$ because the broker keeps the client loop continuously busy — both converge to around 20 ms.

3.2 Normal 5G conditions

Table 2: RTT under normal 5G (2000 B payload, 10 Hz)

Protocol	N	Loss (%)	p50 (ms)	p95 (ms)	p99 (ms)
Zenoh-QUIC	10	1.1	41.69	44.22	45.66
Zenoh (TCP)	10	1.4	42.21	44.78	46.11
MQTT-QUIC	10	0.3	45.32	52.70	66.68
gRPC	10	11.1	43.04	46.11	49.19
MQTT (TCP)	10	2.7	90.48	158.10	196.04
Zenoh-QUIC	20	1.2	41.00	43.71	45.01
Zenoh (TCP)	20	1.8	41.58	44.33	45.61
MQTT-QUIC	20	0.3	45.39	55.68	65.16
gRPC	20	54.9	42.67	45.84	47.78
MQTT (TCP)	20	1.5	460.76	792.57	933.33

At $N = 10$, MQTT-QUIC is now competitive with gRPC and Zenoh — sitting at 45 ms median, right on the 40 ms network floor. Compare that to MQTT TCP still at 90 ms, carrying the network delay plus the broker overhead.

At $N = 20$, two things stand out:

- **gRPC drops 55% of messages.** The messages that do get through have normal latency (~ 43 ms), but more than half never arrive. This is HTTP/2 stream exhaustion under simultaneous fan-out to 20 devices.

- **MQTT TCP is already at 461 ms median** with only 0.1% packet loss. At 20 devices and 10 Hz, cumulative broker load starts outrunning the drain rate. This will cascade into full collapse under any real packet loss.

Zenoh (TCP and QUIC) and MQTT-QUIC scale cleanly to $N = 20$ with sub-2% loss and latency within 5 ms of the network floor.

Figure 1 shows the p95 latency versus device count. The left panel shows all five protocols so the MQTT TCP divergence is visible in context; the right panel zooms into the four that actually work, where you can see the real differences between them.

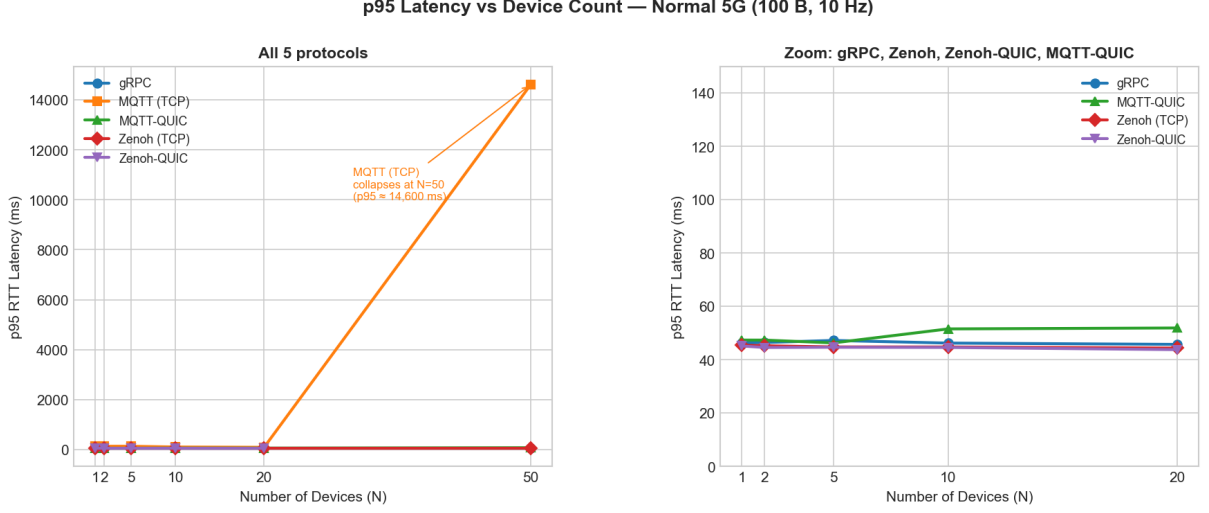


Figure 1: p95 latency vs. device count under normal 5G (100 B, 10 Hz). Left: all five protocols — MQTT TCP diverges at $N = 20$ and saturates at $N = 50$ ($p_{95} \approx 14,600$ ms). Right: zoom on the four viable protocols — Zenoh, Zenoh-QUIC, and gRPC are flat; MQTT-QUIC rises slightly at higher N but stays well within bounds.

Figure 2 shows the same data as a side-by-side snapshot at $N = 20$, with exact p50 values annotated and p95 shown as error bars. The degraded panel uses a log scale so the MQTT TCP bar (19.9 s) and the other protocols (~ 160 – 183 ms) are both visible.

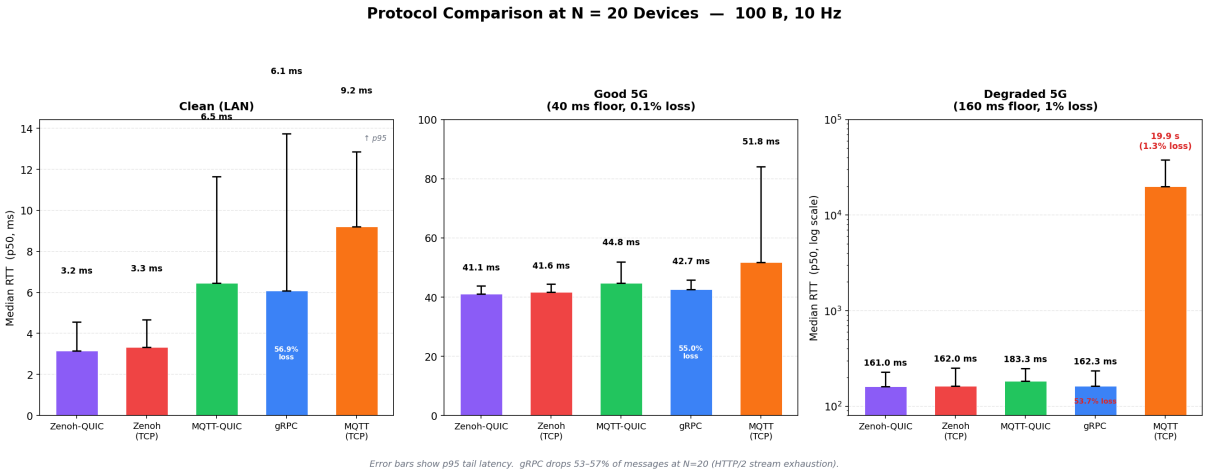


Figure 2: Protocol comparison at $N = 20$, 100 B, 10 Hz. Bars show median RTT (p50); error bars show p95. Left: clean LAN. Centre: normal 5G. Right: degraded 5G (log scale). gRPC loss labels are shown in red inside the bars. MQTT TCP is 19.9 s median under degraded 5G while all other protocols stay below 200 ms.

3.3 Degraded 5G — where the architectures separate

Table 3: RTT under degraded 5G (2000 B payload, 10 Hz)

Protocol	N	Loss (%)	p50 (ms)	p95 (ms)	p99 (ms)
Zenoh-QUIC	10	1.0	163.75	306.65	432.76
Zenoh (TCP)	10	1.4	163.22	262.93	438.20
MQTT-QUIC	10	0.3	210.13	371.26	486.05
gRPC	10	8.2	164.61	229.71	433.40
MQTT (TCP)	10	1.4	38,759	64,452	67,034
Zenoh-QUIC	20	1.0	161.50	254.37	426.78
Zenoh (TCP)	20	1.7	161.88	255.87	425.31
MQTT-QUIC	20	0.3	183.33	249.46	318.23
gRPC	20	53.7	162.13	226.50	427.80
MQTT (TCP)	20	2.2	86,589	166,654	175,299

MQTT-QUIC at $N = 10$ registers a **210 ms median** under degraded 5G — compared to 38,759 ms for MQTT TCP under identical conditions. That is a **185 $\times$ reduction** just from swapping the transport layer.

The root cause is the same one I described in the first report: TCP head-of-line blocking stalls the broker’s PUBACK handshake and the queue grows without bound. QUIC eliminates this because each device’s stream is independent — a lost packet on one stream doesn’t block any other. The broker queue drains continuously.

At $N = 20$, MQTT TCP median reaches **86.6 seconds** even though only 2.2% of messages are lost. The queue bloat compounds with device count.

Figure 3 shows the median latency scaling under degraded conditions, and Figure 4 isolates the MQTT TCP vs MQTT-QUIC comparison directly.

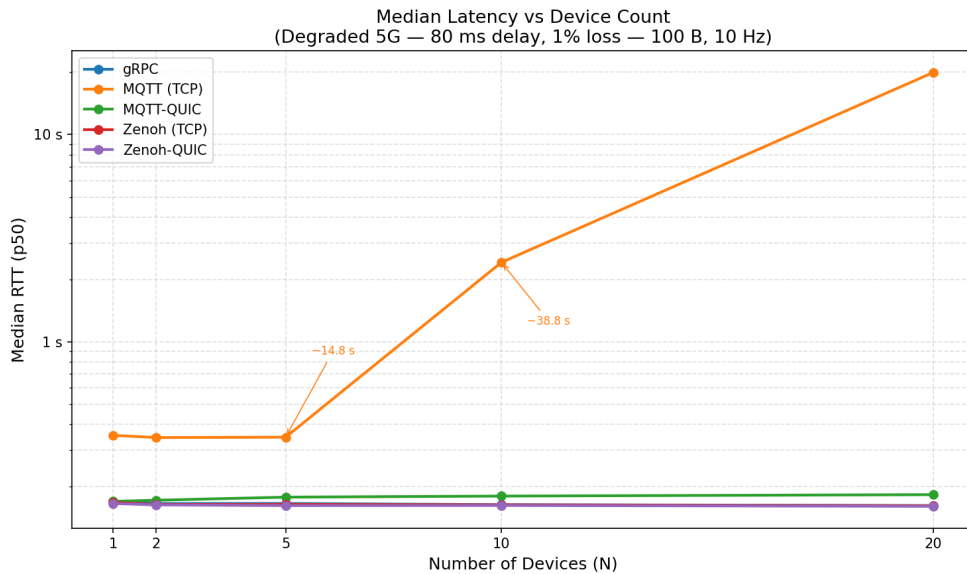


Figure 3: Median RTT vs device count under degraded 5G (100 B, 10 Hz, log scale). MQTT TCP stays near the ~ 350 ms floor through $N = 5$, then collapses — 2.4 s at $N = 10$ and 19.9 s at $N = 20$. All other protocols stay flat between 160 ms and 250 ms across $N = 1$ to $N = 20$.

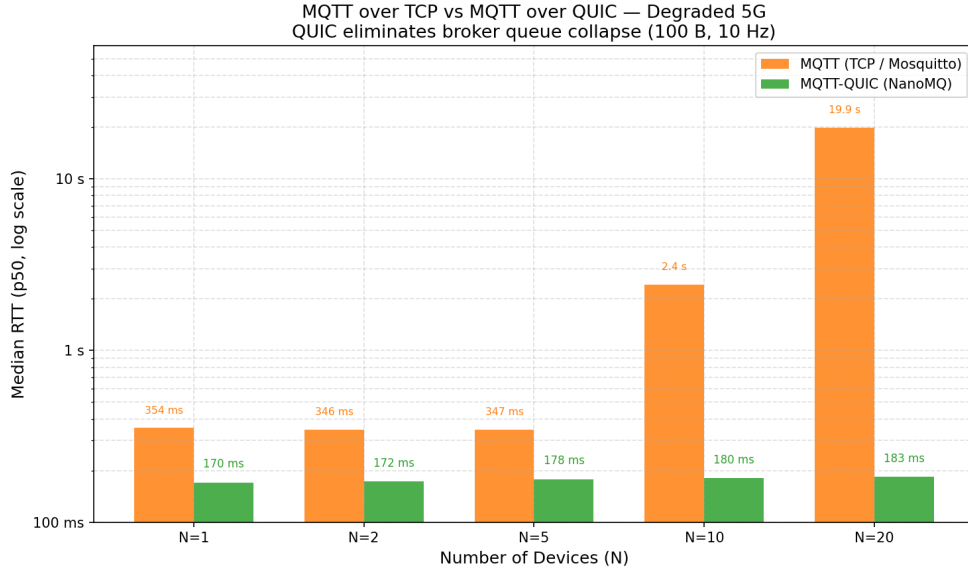


Figure 4: MQTT TCP vs MQTT-QUIC under degraded 5G (100 B, 10 Hz). Swapping the transport from TCP to QUIC, with no other protocol changes, eliminates the queue collapse entirely.

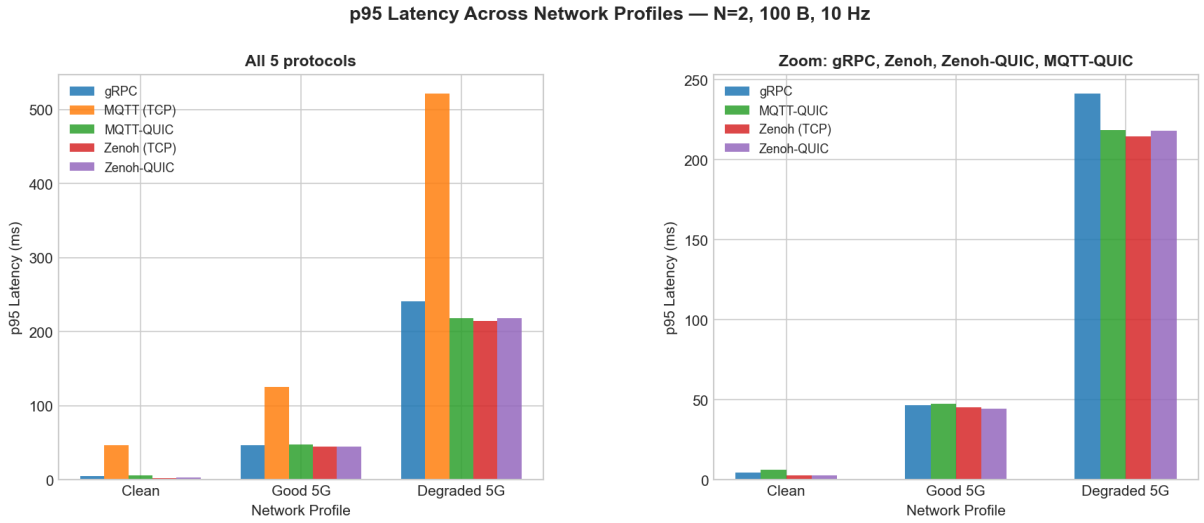


Figure 5: p95 latency across all three network profiles ($N = 2$, 100 B, 10 Hz). Left: all five protocols. Right: zoom on the four viable ones — under degraded 5G, Zenoh TCP, Zenoh-QUIC, and MQTT-QUIC cluster tightly at ~ 215 ms, while gRPC is slightly higher at ~ 241 ms.

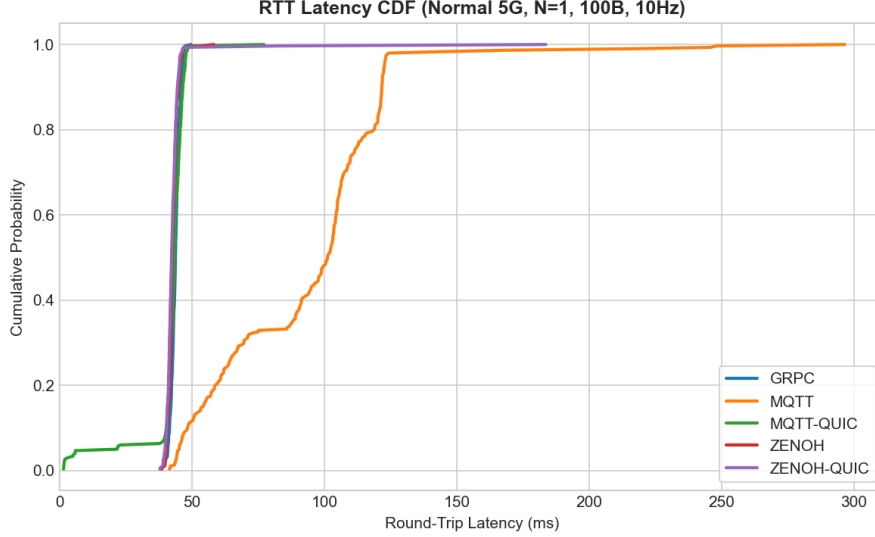


Figure 6: RTT latency CDF under normal 5G ($N = 1$, 100 B, 10 Hz). MQTT TCP is shifted right by ~ 45 ms. Zenoh, Zenoh-QUIC, and MQTT-QUIC all cluster at the 43 ms network floor.

4 What this means for CAMPUS

Table 4 summarises the five protocols at $N = 10$, which is the most representative scale for the current lab setup.

Table 4: Protocol comparison at $N = 10$, 2000 B payload, 10 Hz

Protocol	Clean p50	Normal 5G p50	Degraded p50	Msg loss	Scales to $N = 20$?
Zenoh-QUIC	2.7 ms	42 ms	164 ms	<2%	Yes
Zenoh (TCP)	2.6 ms	42 ms	163 ms	<2%	Yes
MQTT-QUIC	4.9 ms	45 ms	210 ms	<1%	Yes
gRPC	3.3 ms	43 ms	165 ms	8–11%	No (55% loss at $N = 20$)
MQTT (TCP)	44 ms	90 ms	38,759 ms	1–3%	No (queue collapse)

Based on these results, my recommendations are:

1. **Don't use MQTT over TCP for any real-time loop.** It was already disqualified in the first report and the scaling results confirm it. At $N = 20$ under degraded conditions, the median latency is 86 seconds.
2. **Zenoh-over-QUIC is the primary recommendation for the pub/sub layer.** Lowest baseline overhead, flat latency to $N = 20$, best performance under both normal and degraded conditions, and the topic-based filtering maps directly to the ROI-based selective downlink we need for CAMPUS.
3. **MQTT-over-QUIC is the right choice if a central broker is needed.** For example, if the Torino integration requires EMQX or a standard broker for CAMARA API compatibility. QUIC keeps it viable: 210 ms median at $N = 10$ under degraded 5G, clean scaling to $N = 20$.
4. **gRPC for point-to-point transfers only.** HD map streaming, bulk uploads from UE to edge — anything where a direct connection with flow control makes sense. Not suitable as a broadcast protocol.

5 Limitations and next steps

- The network impairment is still synthetic (`tc netem` on a Docker bridge). The next step is to run this on the lab’s physical 5G testbed to validate the protocol rankings hold with real radio. I’m planning to start with Open5GS + UERANSIM to get familiar with the 5G deployment workflow before moving to the physical setup.
- Each configuration was run once for 30 seconds. For a publication-quality result, I’d run each 3–5 times and report median with confidence intervals.
- I didn’t profile CPU or memory consumption of the containers. QUIC processing happens in user space and may carry higher CPU overhead than kernel-space TCP at production scale — this is unmeasured.
- gRPC here is still HTTP/2 over TCP; I did not test gRPC over HTTP/3 (QUIC). The QUIC transport benefit is already isolated in these results through the Zenoh-TCP→Zenoh-QUIC and MQTT-TCP→MQTT-QUIC comparisons (same protocol, transport swapped), so a third QUIC stack would mainly re-confirm the same mechanism. It is also non-trivial: mainstream gRPC has no native HTTP/3 support and would require an experimental build or a QUIC-terminating proxy. I’d hold gRPC-over-HTTP/3 as a focused transport-only study for later, rather than expand the current matrix and mix the transport and architecture variables this design keeps separate.
- At $N = 50$, several Zenoh and Zenoh-QUIC runs returned no data (header-only CSV files). The root cause was not isolated — likely WSL2 disk exhaustion or a concurrency race condition in the test harness, rather than a Zenoh protocol limit, since the failure pattern was inconsistent across runs and container resource usage was not profiled. The $N \leq 20$ data is clean and sufficient for the architecture decision; $N = 50$ should be revisited on the physical testbed.